

ExoHabitAI: Deployment Documentation

Overview

This document outlines the infrastructure and deployment pipeline for **ExoHabitAI**. The application is deployed as a decoupled full-stack architecture on **Render**, consisting of a Python/Flask Machine Learning API and a React/Vite 3D Single Page Application (SPA).

Part 1: Backend Deployment (Flask API)

The backend acts as the "Brain" of the application, serving the Random Forest ML model and processing telemetry data. It is deployed as a dynamic **Web Service**.

1. Render Configuration

- **Service Type:** Web Service
- **Root Directory:** `backend`
- **Runtime:** Python
- **Build Command:** `pip install -r requirements.txt`
- **Start Command:** `gunicorn app:app` (*Gunicorn is strictly required for production, replacing the built-in Flask development server*).

2. Environment Variables & Troubleshooting

During deployment, modern Python environments (3.13+) may lack the C++ compilers needed to build heavy data science libraries like `pandas` and `scikit-learn` from source, resulting in a `metadata-generation-failed` error.

To resolve this, we forced Render to use a highly stable Python version that supports pre-compiled wheel files:

- **Key:** `PYTHON_VERSION`
 - **Value:** `3.11.10`
-

Part 2: Frontend Deployment (React / Vite)

The frontend serves the 3D dynamic planetary interface. Because Vite compiles React into static HTML/CSS/JS, it does not require a runtime server and is deployed to Render's global CDN as a **Static Site**.

1. Render Configuration

- **Service Type:** Static Site
- **Root Directory:** `frontend`
- **Build Command:** `rm -rf node_modules package-lock.json && npm install && npm run build`
- **Publish Directory:** `dist`

2. Resolving Build Conflicts

The custom build command above is critical for two reasons:

1. **NPM ERESOLVE Conflicts:** It forces a clean installation to bypass peer dependency conflicts between `@react-three/drei` and `@react-three/fiber`.
2. **Rollup Linux Bug (EBADPLATFORM):** Because development occurs on Windows but Render operates on Linux, caching the `node_modules` causes Vite's bundler (Rollup) to crash looking for the `@rollup/rollup-linux-x64-gnu` binary. Deleting the lockfile and modules ensures Render downloads the correct Linux-specific files during deployment.

3. SPA Routing (Fixing 404 Errors)

Because React is a Single Page Application, refreshing any page other than the root (`/`) causes a 404 error on standard static servers. To fix this, a **Rewrite Rule** was added to Render's settings:

- **Source:** `/*`
- **Destination:** `/index.html`
- **Action:** `Rewrite`

Part 3: Connecting the Frontend and Backend

To bridge the decoupled architecture, the React frontend must be explicitly pointed to the live Flask API URL.

1. Updating API Endpoints

In the frontend source code (e.g., `Analyzer.jsx` and `Dashboard.jsx`), local API calls were replaced with the production URL:

JavaScript

// Local Development

```
const res = await axios.post('http://127.0.0.1:5000/predict', formData);
```

// Production Environment

```
const res = await axios.post('https://exohabitai-backend.onrender.com/predict', formData);
```

2. Environment Variables

For future scalability, API connections have been handled via Vite environment variables.

1. Create a `.env` file in the `frontend` directory:

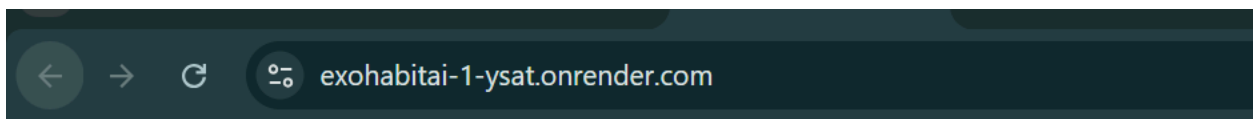
```
VITE_API_URL=https://exohabitai-backend.onrender.com
```

2. Update the Axios requests to call the variable:

```
axios.post(`${import.meta.env.VITE_API_URL}/predict`, formData);
```

Current Status

- **Backend:** Successfully receiving telemetry via POST requests and returning JSON predictions
- **.Link:** <https://exohabitai-1-ysat.onrender.com>



 ExoHabitAI Backend API is Live and Running!

- **Frontend:** Successfully fetching data, mutating 3D models via `three.js`, and routing traffic seamlessly.
- **Link:** <https://exohabitai-ui.onrender.com>

